

Programowanie Obiektowe – Kolokwium 2

Data: 15.06.2025 r. | Czas trwania: 90 minut



Zasady i organizacja kolokwium

Podczas kolokwium wszystkie działania wykonywane na ekranie komputera są rejestrowane i mogą zostać odtworzone przez prowadzących. Po napisaniu kolokwium, katalog z projektem należy spakować do formatu .zip lub .tar.gz i wysłać go przy użyciu platformy Kampus. Proszę nie wyłączać komputera po wysłaniu zadania.

Podczas kolokwium można korzystać ze wszystkich zasobów internetu, w tym z samodzielnie przygotowanych materiałów, z poniższym zastrzeżeniem:



UWAGA: Bezwzględnie zakazane jest korzystanie ze stron, aplikacji i pluginów opartych o sztuczną inteligencję (AI). Podczas kolokwium zabroniona jest wzajemna komunikacja oraz publikowanie i dostęp do kodu stworzonego na potrzeby kolokwium przez inne osoby. Zabronione jest korzystanie z innych urządzeń niż udostępniony komputer stacjonarny.

Przedstawione poniżej kroki stanowią propozycję kolejności rozwiązywania zadania. Po przeczytaniu całości można zdecydować o rozwiązywaniu w innej kolejności.



Wprowadzenie do zadania

Urząd Lotnictwa Cywilnego na bieżąco monitoruje ruch powietrzny. Każdego miesiąca agregowane są dane dotyczące liczby lotów, opóźnień oraz odwołań na poszczególnych trasach dla konkretnych linii lotniczych. Dane te publikowane są w formacie CSV z podziałem na kraj, lotnisko startowe oraz lotnisko docelowe. Przygotowano pliki z danymi za styczeń i luty bieżącego roku. Napisz aplikację, która umożliwi analizę tych danych, odnajdywanie najbardziej problematycznych przewoźników oraz wizualizację statystyk.



Kroki do wykonania

Krok 1. ☆☆☆

Utwórz rekord `Candidate` (lub w tym kontekście `Airline`) zawierający jedno pole tekstowe – nazwę linii lotniczej.

Krok 2. ★☆☆

Utwórz klasę `TrafficSystem`, w której zdefiniujesz prywatne pole `airlines` – listę obiektów `Airline`. Dodaj do niej metodę akcesorową, która zwróci nową listę będącą bezpieczną kopią oryginalnej listy `airlines` (zachowując te same referencje do obiektów wewnątrz listy).

Krok 3. ★☆☆

W klasie `TrafficSystem` dodaj metodę `populateAirlines()`, która przyjmie jako argument ścieżkę do pliku z liniami lotniczymi i wczyta z niego dane, zapisując obiekty do listy `airlines`. Zaimplementuj także metodę `initialize()`, która wywoła `populateAirlines()` i będzie dalej rozbudowywana w kolejnych krokach.

Krok 4. ★☆☆

Utwórz klasę `FlightMonth`, której konstruktor przyjmie listę obiektów `Airline` i zapisze ją w polu klasy. Następnie w klasie `TrafficSystem` dodaj pola `januaryData` i `februaryData` typu `FlightMonth`. Obiekt styczniowy należy stworzyć od razu, obiekt lutowy początkowo powinien być nullem. Napisz odpowiednie gettery.

Krok 5. ★☆☆

Utwórz klasę `RouteRecord`, która zawiera:

- Prywatną mapę `delaysByAirline`, w której kluczem jest obiekt `Airline`, a wartością – obiekt klasy osłonnej `Integer` (liczba opóźnień).
- Prywatną listę `routeDetails`, przechowującą w kolejności: Kraj, Lotnisko Startowe, Lotnisko Docelowe.

Krok 6. ★☆☆

W klasie `RouteRecord` dodaj publiczną, statyczną metodę `fromCsvLine()`, która:

1. Przyjmuje wiersz tekstu z pliku CSV.
2. Parsuje zawartość tego wiersza i tworzy nowy obiekt `RouteRecord`.
3. Wypełnia struktury danych utworzone w Kroku 5.
4. **Wyjątki:** Jeśli wiersz zawiera błędy (np. brakującą kolumnę lub tekst zamiast liczby przy opóźnieniach), metoda powinna wyrzucić samodzielnie zdefiniowany wyjątek `CorruptedDataException`.

Krok 7. ★★☆☆

W klasie `FlightMonth`:

- Dodaj prywatne pole `records`, będące listą obiektów `RouteRecord`.
- Zaimplementuj metodę `loadData(String path)`, która wczyta dane z pliku CSV i zapełni nimi listę `records` (pomiń błędne wiersze przechwytyjąc wyjątek z Kroku 6 i wypisując ostrzeżenie na konsolę).
- Następnie w klasie `TrafficSystem`, w metodzie `initialize()`, wywołaj `januaryData.loadData()`.

Krok 8. ★★☆☆

W klasie `RouteRecord` dodaj metodę `aggregate()`, która:

1. Przyjme jako argument listę obiektów `RouteRecord`.
2. Utworzy i zwróci nowy obiekt `RouteRecord`, którego mapa `delaysByAirline` będzie zawierać sumy opóźnień z poszczególnych tras dla każdej linii lotniczej.
3. Ustawi pole `routeDetails` w nowym obiekcie jako pustą listę.

Krok 9. ★☆☆☆

W klasie `RouteRecord` dodaj metody przyjmujące linię lotniczą jako argument:

- `delays(Airline a)` – zwraca liczbę opóźnień dla danej linii.
- `delayShare(Airline a)` – zwraca ułamek (lub procent) opóźnień tej linii w stosunku do łącznej liczby opóźnień wszystkich linii na danej trasie/agregacji.

Krok 10. ★☆☆☆

W klasie `RouteRecord` nadpisz metodę `toString()`, tak aby zwracała napis zawierający w kolejnych liniach nazwy linii lotniczych oraz ich procentowy udział w opóźnieniach na danej trasie. Przetestuj działanie w metodzie `main()`.

Krok 11. ★★☆☆

Aby wygenerować napis w `toString()`, wielokrotnie potrzebujesz sumy wszystkich opóźnień z obiektu `RouteRecord`. Zastosuj mechanizm "leniwej inicjalizacji" (lazy initialization) oraz enkapsulację, aby zagwarantować, że suma ta będzie obliczona tylko jeden raz w cyklu życia obiektu, a jej wyliczenie nastąpi dopiero przy pierwszym odwołaniu.

Krok 12. ★★☆☆

W klasie `FlightMonth` zaimplementuj metodę `worstAirline()`, która zwraca linię lotniczą (obiekt `Airline`) o największej sumarycznej liczbie opóźnień w danym miesiącu. Jeśli z jakiegoś powodu suma wszystkich opóźnień dla całego miesiąca wynosi 0, metoda powinna rzucić wyjątek `NoDelaysException`.

Krok 13. ★☆☆☆

W klasie `TrafficSystem` dodaj prywatne pole `mostDelayed`, przechowujące referencję do najgorzej punktującej linii. W metodzie `initialize()` wywołaj `worstAirline()` na obiekcie `januaryData` i zapisz wynik.

Krok 14. ★★☆☆

Utwórz generyczny interfejs `DataFilter<T>` z metodą `boolean test(T item)`. Zastosuj mechanizmy programowania generycznego. Zaimplementuj w dowolnym miejscu programu anonimową klasę lub wyrażenie lambda implementujące ten interfejs w taki sposób, by sprawdzał, czy dany lot miał więcej niż 100 opóźnień.

Krok 15. ★☆☆☆

Jeśli w Kroku 13 wywołanie `worstAirline()` rzuci wyjątek (brak opóźnień w styczniu, np. loty były uziemione z powodu złej pogody), w bloku `catch` należy:

1. Zainicjalizować obiekt `februaryData`.
2. Wczytać dane z pliku `luty.csv`.
3. Ustawić najgorzej punktującą linię z lutego jako główną wartość pola `mostDelayed` w klasie `TrafficSystem`.

Krok 16. ★★★

Wykorzystując **Stream API**, w klasie `RouteRecord` zaimplementuj publiczną, statyczną metodę `filterByRoute()`, która przyjmuje dwa argumenty: listę obiektów `RouteRecord` oraz listę napisów opisujących poziom trasy. Metoda ma zwrócić odfiltrowaną listę przy użyciu strumieni. Filtrowanie zależy od długości drugiej listy:

- 1 element (np. "Polska") – zwraca trasy tylko z tego kraju.
- 2 elementy (np. "Polska", "Lotnisko Chopina") – zwraca loty z kraju z konkretnego lotniska startowego.
- 3 elementy (np. "Polska", "Lotnisko Chopina", "JFK") – zwraca konkretną relację.

Krok 17. ★☆☆

Zastosuj statyczny polimorfizm. W klasie `FlightMonth` utwórz dwie wersje metody `aggregateData()`:

- Bezargumentowa: agreguje (wykorzystując metodę z kroku 8) całą listę `records`.
- Z argumentem `List<String> routeFilter`: najpierw filtruje listę używając metody z kroku 16, a dopiero potem agreguje wynik.

Krok 18. ★☆☆

Zapoznaj się z dostarczoną klasą `ChartGenerator`. Znajduje się w niej metoda `saveToHtml()`, która na podstawie dostarczonych danych generuje prosty plik HTML ze słupkowym wykresem opóźnień. Przetestuj jej wywołanie w `main()`.

Krok 19. ★☆☆

W nowej klasie analizującej, stwórz metodę opartą w 100% o Stream API, która z listy rekordów wyciągnie listę unikalnych lotnisk startowych (użyj mapowania i metody `distinct()`). Wygeneruj w `main()` zagregowane statystyki opóźnień osobno dla każdego z 3 pierwszych lotnisk startowych.

Krok 20. ★☆☆

Stwórz klasę `ReportChart` dziedziczącą (Inheritance) po klasie abstrakcyjnej `ChartGenerator` (zmodyfikuj ją tak, aby była klasą abstrakcyjną). Dodaj w klasie bazowej abstrakcyjną (Abstraction) metodę `String determineColor(Airline a)`, która decyduje o kolorze słupka na wykresie HTML. Zaimplementuj tę metodę w podklasie tak, aby linia lotnicza, na rzecz której oddano najwięcej skarg (wyłoniła w krokach 12-15) była rysowana kolorem czerwonym ("`#FF0000`"), a pozostałe szarym ("`#CCCCCC`").

Krok 21. ★★★

Stwórz klasę `DelayHeatMap`, dziedziczącą po `ChartGenerator`. Skonstruuj ją tak, by przyjmowała zagregowane wyniki z Kroku 19 i generowała zbiorczy plik HTML (lub tekstowy). Plik ma zawierać zestawienie: nazwa lotniska, a obok nazwa przewoźnika z najgorszym wynikiem opóźnień startujących z tej lokalizacji. Przetestuj działanie całego ekosystemu w metodzie `main()`.

Załączniki i struktura plików

Struktura pliku `linie_lotnicze.txt`

```
LOT Polish Airlines
Lufthansa
Ryanair
WizzAir
Emirates
KLM
```

Struktura plików CSV (`styczen.csv` / `luty.csv`)

```
Kraj, LotniskoStartowe, LotniskoDocelowe, LOT Polish
Airlines, Lufthansa, Ryanair, WizzAir, Emirates, KLM
Polska, Lotnisko Chopina, Heathrow, 120, 45, 0, 0, 12, 34
Polska, Lotnisko Chopina, CDG, 80, 102, 15, 40, 0, 55
Polska, Balice, Stansted, 0, 10, 210, 180, 0, 0
Niemcy, Frankfurt, Lotnisko Chopina, 40, 300, 5, 0, 0, 0
```